

LECTURE SCRIPT

PPO

1 PG Recap and REINFORCE

Consider an episodic Markov decision process with horizon H ($H = \infty$ works analogously, as seen in the RL lecture) and a differentiable stochastic policy $\pi_\theta(a; s)$. At time t , the agent observes S_t , chooses A_t , receives R_t , and moves to S_{t+1} . For a finite-horizon problem, the state includes time, so we can write $V^\pi(S_t)$ without a separate time index. Episodes that finish early can be padded by an absorbing state with zero reward. The objective and the reward-to-go are

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^H \gamma^t R_t \right], \quad G_t = \sum_{k=t}^{H-1} \gamma^{k-t} R_k, \quad 0 < \gamma \leq 1. \quad (1)$$

Policy Gradient algorithms hinge on the policy gradient theorem, which gives the gradient of the objective in terms of the policy's score function and the reward-to-go:

$$\nabla J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{H-1} \gamma^t \nabla_\theta \log \pi_\theta(A_t; S_t) G_t \right]. \quad (2)$$

The discount factors in the front and inside the reward-to-go have different roles: G_t discounts relative to *time* t , while the outer γ^t discounts relative to the *start of the episode*.

The REINFORCE update

Collect B independent episodes under the current policy and form

$$\widehat{\nabla}_\theta J(\theta) = \frac{1}{B} \sum_{i=1}^B \sum_{t=0}^{H-1} \gamma^t \nabla_\theta \log \pi_\theta(a_t^i; s_t^i) G_t^i, \quad \theta \leftarrow \theta + \alpha \widehat{\nabla}_\theta J(\theta). \quad (3)$$

This is stochastic gradient *ascent*: actions followed by high returns receive a positive push in log probability. Under the usual regularity assumptions, the estimator is unbiased for (1).

Algorithm 1: Vanilla REINFORCE

Initialize policy parameters θ ; choose batch size B and step size α .

repeat

 Collect B complete episodes using the current policy π_θ .

for each episode i **do**

 Set $G_H^i \leftarrow 0$; compute $G_t^i \leftarrow r_t^i + \gamma G_{t+1}^i$ backwards.

 Form the batch gradient $\widehat{\nabla}_\theta J(\theta)$ in (3) using these returns.

 Update $\theta \leftarrow \theta + \alpha \widehat{\nabla}_\theta J(\theta)$; discard the batch.

until the training budget is exhausted.

Two problems determine the route to PPO

Variance: G_t contains randomness from many later decisions and transitions. Long episodes make gradient estimates very noisy.

Sample efficiency: after an update, the rollouts are discarded. In real-life examples, what is expensive (real-time execution, damages from bad rollouts,...) is the cost of generating new data.

2 A2C: Advantage Actor-Critic

Define the state-action value function and the state value function of a policy by

$$Q_t^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a], \quad V_t^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot \mid s)}[Q_t^\pi(s, a)]. \quad (4)$$

Starting from the policy gradient theorem, we obtain

$$\begin{aligned} \nabla J(\theta) &= \sum_{t=0}^{H-1} \gamma^t \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(A_t \mid S_t) G_t] = \sum_{t=0}^{H-1} \gamma^t \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(A_t \mid S_t) Q_t^{\pi_\theta}(S_t, A_t)] \\ &= \sum_{t=0}^{H-1} \gamma^t \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(A_t \mid S_t) (Q_t^{\pi_\theta}(S_t, A_t) - b(S_t))] \\ &= \sum_{t=0}^{H-1} \gamma^t \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(A_t \mid S_t) \underbrace{(Q_t^{\pi_\theta}(S_t, A_t) - V_t^{\pi_\theta}(S_t))}_{:= A_t^{\pi_\theta}(S_t, A_t)}]. \end{aligned}$$

Here, we used the tower property in the first equation, the baseline trick from the RL lecture in the second, and the value function as baseline in the third. Conditioning removes future-return sampling noise from each gradient term. Subtracting the value baseline then measures an action relative to the policy's usual return in the same state, removing variation due to how favorable that state is. This typically reduces gradient variance without changing its mean; the value baseline is not necessarily the variance-minimizing baseline. The differences $A_t^{\pi_\theta}(S_t, A_t)$ are called *advantages*.

Actor-critic Recap and A2C

Actor-critic methods alternate two tasks: *policy evaluation* estimates how good the current policy is (the critic), and *policy improvement* changes the policy using that evaluation (the actor). In the RL lecture, we had model-based policy iteration as an example. We alternate between evaluating the current policy by solving its Bellman equations (critic; policy evaluation), then make the policy greedy with respect to that value (actor; policy improvement):

$$\begin{aligned} V^\pi(s) &= \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) [r(s, a, s') + \gamma V^\pi(s')], \\ \pi_{\text{new}}(s) &\in \arg \max_a \sum_{s'} P(s' \mid s, a) [r(s, a, s') + \gamma V^\pi(s')]. \end{aligned} \quad (5)$$

In sample-based actor-critic methods, a learned critic replaces exact evaluation and a general policy gradient step (e.g. through policy gradients) replaces the greedy update. A2C learns V^π by TD learning to evaluate the current policy: this is the *critic*. The policy π_θ takes stochastic gradient ascent steps weighted by estimated advantages: this is the *actor*. Now we only need a method to estimate the advantages from the critic's V^π .

Generalized advantage estimation (GAE)

For fixed critic predictions $\widehat{V}_t$, the one-step and k -step TD errors are

$$\delta_t = R_t + \gamma \widehat{V}_{t+1} - \widehat{V}_t, \quad \widehat{A}_t^{(k)} = \sum_{\ell=0}^{k-1} \gamma^\ell R_{t+\ell} + \gamma^k \widehat{V}_{t+k} - \widehat{V}_t \stackrel{\text{Telescope}}{=} \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}. \quad (6)$$

Short horizons typically reduce variance but rely on a potentially biased bootstrap; longer horizons use more noisy rewards. GAE balances these effects with $\lambda \in [0, 1]$. For m remaining transitions, define

$$\widehat{A}_t^{\text{GAE}} = (1 - \lambda) \sum_{k=1}^{m-1} \lambda^{k-1} \widehat{A}_t^{(k)} + \lambda^{m-1} \widehat{A}_t^{(m)} = \sum_{l=0}^{m-1} (\gamma \lambda)^l \delta_{t+l}. \quad (7)$$

3 Sample reuse

To tackle the problem of sample efficiency, we can reuse previously generated samples. Let $\pi_{\text{old}} = \pi_{\theta_{\text{old}}}$ generate a batch of trajectories. To take several policy updates with these data, importance sampling (IS) changes the sampling distribution from π_{θ} to π_{old} . Assuming sufficient support, the gradient is

$$\nabla J(\theta) = \sum_{t=0}^{H-1} \gamma^t \mathbb{E}_{\pi_{\text{old}}} \left[\left(\prod_{j=0}^t \frac{\pi_{\theta}(A_j | S_j)}{\pi_{\text{old}}(A_j | S_j)} \right) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) A_t^{\pi_{\theta}}(S_t, A_t) \right]. \quad (8)$$

The environment probabilities cancel. Earlier action factors correct the state distribution; the last factor corrects the action distribution at S_t . Here IS permits sample reuse, rather than itself reducing variance.

Three problems with exact sample reuse

Products and fractions: Fractions of probabilities, especially if combined with long products can become very large, giving a high-variance gradient estimate.

Advantage estimation: the formula requires advantages for the current π_{θ} , which can only be estimated with samples from the current policy.

Correlated rollouts: neighboring transitions share history; using whole rollouts as optimization units gives strongly correlated contributions.

How to solve the problems: local weighting, old advantages, random transitions

Keep only the last factor of the product and introduce a trust region $C_t(\theta_{\text{old}})$: the action probability has changed too far in the favorable direction while we still work with stale data. The gate $c_t(\theta) = \mathbf{1}_{\{\theta \notin C_t(\theta_{\text{old}})\}}$ suppresses those contributions, limiting how long we trust the stale data. Replace the current advantage by the old one, then sample individual transitions uniformly. Write $r_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\text{old}}(A_t | S_t)}$ and abbreviate $A_t^{\text{old}} = A_t^{\pi_{\text{old}}}(S_t, A_t)$. For $U \sim \text{Unif}\{0, \dots, H-1\}$,

$$\begin{aligned} \nabla J(\theta) &\approx \sum_{t=0}^{H-1} \gamma^t \mathbb{E}_{\pi_{\text{old}}} \left[\frac{\pi_{\theta}(A_t | S_t)}{\pi_{\text{old}}(A_t | S_t)} c_t(\theta) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) A_t^{\text{old}} \right] \\ &= \sum_{t=0}^{H-1} \gamma^t \mathbb{E}_{\pi_{\text{old}}} \left[\frac{\nabla_{\theta} \pi_{\theta}(A_t | S_t)}{\pi_{\text{old}}(A_t | S_t)} c_t(\theta) A_t^{\text{old}} \right] \\ &= H \mathbb{E}_{U, \pi_{\text{old}}} \left[\gamma^U \nabla_{\theta} r_U(\theta) c_U(\theta) A_U^{\text{old}} \right]. \end{aligned}$$

The second line uses $\pi \nabla \log \pi = \nabla \pi$; the third rewrites a sum as a uniform expectation. In a buffer, replace A_t^{old} by its fixed GAE estimate $\hat{A}_t$ and mix transitions from all rollouts in minibatches. Shuffling mixes correlated observations; it does not make the original data independent.

We deliberately trade **bias in the gradient** for lower variance and the ability to reuse expensive samples. Keeping the policy near the version that collected the data makes old advantages more relevant. The precise gate is the PPO clipping rule below. A more standard motivation comes from trust-region policy optimization, which we will likely encounter in a later lecture.

Common PPO losses omit the outer γ^t . To retain it, either weight uniform buffer samples by γ^t , or draw transitions with probability proportional to γ^t (up to a constant normalization), as in the second lecture's homework. Below, $w_t = \gamma^t$ denotes the first option; $w_t = 1$ denotes the usual uniform PPO convention. Here t is episode time.

4 PPO: Proximal Policy Optimization

The choice of trust region

Initially $r_t(\theta) = r_t(\theta_{\text{old}}) = 1$. The sign of the stored advantage determines which change is favorable.

- If $\hat{A}_t > 0$, increase the probability of A_t . Once $r_t > 1 + \epsilon$, stop rewarding further increases. If the probability instead decreases, keep the gradient that pushes it back up.
- If $\hat{A}_t < 0$, decrease the probability of A_t . Once $r_t < 1 - \epsilon$, stop rewarding further decreases. If the probability instead increases, keep the gradient that pushes it back down.

The asymmetric clipping is so that information from opposite advantages is not lost.

The PPO loss

The derivation gives an improvement direction in the form of a gradient. The neural networks used for deep reinforcement learning cannot generally realize arbitrary changes to individual action probabilities, or be optimized in one exact step. Instead, we specify a differentiable loss and let a numerical optimizer take parameter steps on minibatches. Its computed gradients are those of the sampled loss, rather than the exact expected return. We will study network training and these optimizers (especially for the critic using TD-Learning) more closely in later lectures. We obtain the classical PPO loss:

The PPO Loss

$$L^{\text{clip}}(\theta) = \mathbb{E}_{U, \pi_{\text{old}}} \left[w_U \min \left\{ r_U(\theta) \hat{A}_U, \text{clip}(r_U(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_U \right\} \right]. \quad (9)$$

Algorithm 2: PPO

Initialize policy parameters θ (actor), value function V (critic);

choose batch size B , number of epochs E , and step size α .

repeat

Collect B complete episodes using the fixed policy π_θ and compute advantages backwards.

Draw E minibatches of size M from the buffer.

Alternately update actor $\theta \leftarrow \theta + \alpha \widehat{\nabla}_\theta J(\theta)$ and critic V (TD learning) on each minibatch.

until the training budget is exhausted.

A few differences in practice

- **Shared networks:** actor and critic may share parameters. Then use a combined actor and value-prediction loss and a joint optimizer step, rather than two separate updates as in the tabular standard algorithm.
- **Finite and infinite horizons:** a rollout is necessarily a finite segment. Bootstrap at nonterminal cutoffs from the final observation before any reset; use zero value at true termination.
- **Adam versus SGD:** implementations often use Adam (or other more sophisticated optimizers) in place of SGD steps; it adapts steps using gradient history.
- **Unequal rollout lengths:** episodes or collected segments may have different lengths. The buffer then contains $\sum_i T_i$ transitions; sample transitions, and compute GAE separately within each segment.
- **Gamma** γ is often treated as a hyperparameter in practice, allowing for tuning based on the specific task and environment. It is also common, as noted, to not use γ in the outer weighting of the PPO loss.